





**Facultad de Ciencias Físico Matemáticas**

**Algoritmia y Programación**

Tarea 2:
Ejercicios

Docente: JOSE ANASTACIO HERNANDEZ SALDAÑA

Saúl García Rivera 2154486

Grupo 031

24/10/2026

**Ejercicio 1. Aceleración Paralela (Paralel Spedup)**

El cálculo de π mediante el método de Monte Carlo es un algoritmo *embarrassingly parallel* (avergonzadamente paralelo), ya que cada muestra generada es estadísticamente independiente de las demás. En Python, debido al *Global Interpreter Lock* (GIL) que limita el multithreading para tareas dependientes de CPU, se utiliza el módulo multiprocessing para instanciar múltiples procesos independientes en el espacio de usuario.

La aceleración o speedup ($S_p$) cuantifica el factor de reducción de tiempo de pared (*wall-clock time*) al usar $p$ procesos comparado con la ejecución en un solo proceso secuencial ($T_1$):

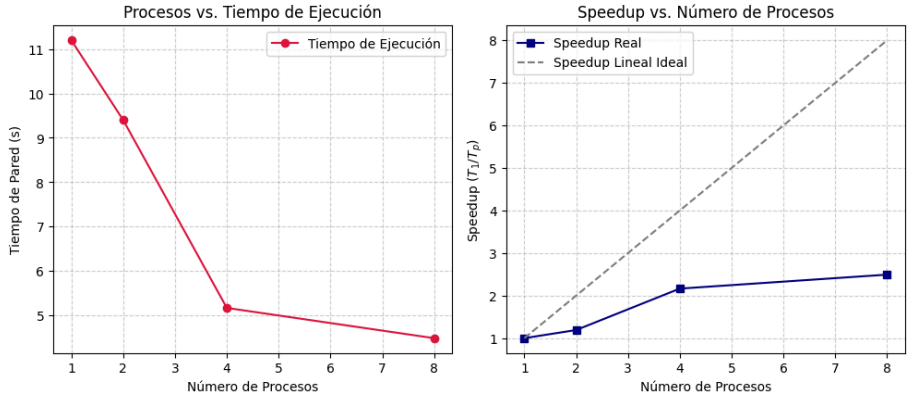
$$S_p = \frac{T_1}{T_p}$$

El speedup lineal ideal establece que $S_p = p$.

Ejecución del código:

```
Procesos: 1 | Tiempo: 11.1938s | Estimación Pi: 3.141696
Procesos: 2 | Tiempo: 9.3996s | Estimación Pi: 3.141633
Procesos: 4 | Tiempo: 5.1708s | Estimación Pi: 3.141629
Procesos: 8 | Tiempo: 4.4902s | Estimación Pi: 3.141543
```

Gráfica Proceso vs tiempo y speedup




**Respuesta Analítica**

La aceleración obtenida en la práctica dista de la línea ideal por tres factores arquitectónicos y algorítmicos principales:

1. **Sobrecarga de gestión y comunicación (Overhead de IPC):** La instanciación del grupo de procesos (mp.Pool), la serialización/deserialización de argumentos mediante *pickle* y la recolección final de resultados en el proceso primario (*Collector Pattern*) introducen latencias fijas inexistentes en la ejecución secuencial pura.
2. **Ley de Amdahl:** La aceleración está estrictamente acotada por la fracción estrictamente secuencial del programa ($f$). Actividades como la asignación de memoria, creación del pool de trabajo y la reducción del acumulador global no son paralelizables.
3. **Contención de Hardware y Recursos Compartidos:** Al aumentar el número de procesos hacia o por encima de los núcleos físicos reales del CPU, se genera competencia por el ancho

de banda del bus de memoria principal, cambios de contexto del sistema operativo y saturación de las memorias caché de último nivel ($L_3$)

**Ejercicio 2: Análisis Algorítmico (Greedy Knapsack)**

**Formulación y Modificación de la Heurística**

El problema de la Mochila $0/1$ (0/1 Knapsack) busca maximizar $\sum_{i=1}^{n} v_i x_i$ sujeto a $\sum_{i=1}^{n} w_i x_i \leq W$, con $x_i \in \{0, 1\}$. Al ser un problema NP-hard, las heurísticas golosas (greedy) toman decisiones mopes e irreversibles basadas en criterios locales.

Se compara el criterio estándar basado en la densidad de valor/peso ($\frac{v_i}{w_i}$) frente al criterio modificado que prioriza objetos de menor peso primero ($w_i$).

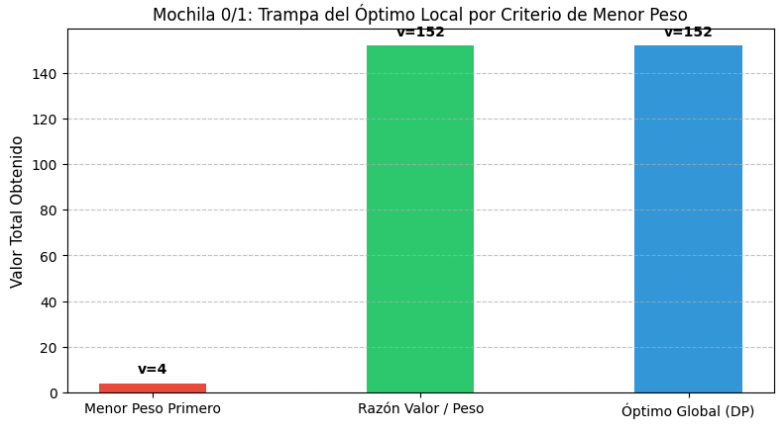
Diseños de la Instancia de Contraejemplo

- Capacidad de la Mochila: $W = 100$
- Conjunto de Objetos Disponibles:
  - Objeto A: $v_A = 2$, $w_A = 10$ => $v_A/w_A = 0.20$
  - Objeto B: $v_B = 2$, $w_B = 10$ => $v_B/w_B = 0.10$
  - Objeto C: $v_C = 150$, $w_C = 90$ => $v_C/w_C = 1.667$

Código:

```
Menor Peso -> Objetos: ['Objeto A', 'Objeto B'] | Peso: 30 | Valor Total: 4
Valor/Peso -> Objetos: ['Objeto C', 'Objeto A'] | Peso: 100 | Valor Total: 152
```

Gráfica:




## Análisis para el Reporte

Una heurística golosa miope que prioriza únicamente el menor peso asume de manera errónea que empacar muchos elementos ligeros maximiza la ganancia total. Al no evaluar la magnitud absoluta del beneficio, la mochila se satura con objetos insignificantes ("basura liviana"), consumiendo espacio y dejando fuera elementos con una recompensa sustancialmente superior que requerían un margen de peso mayor.

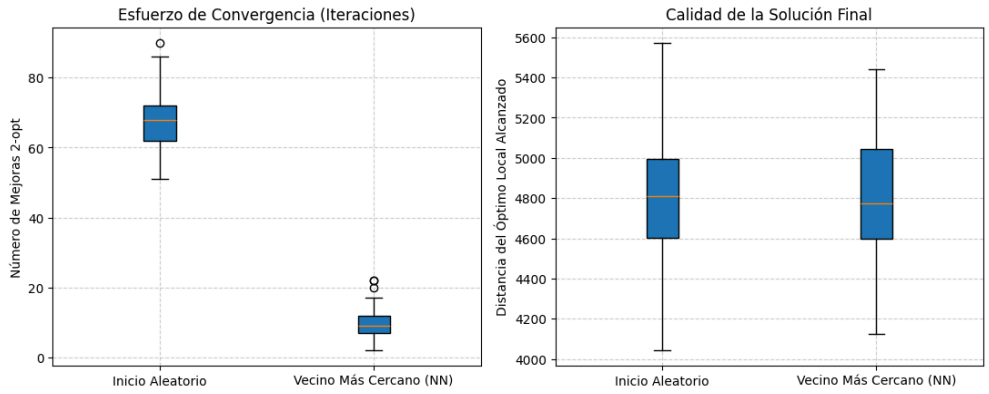
En la instancia presentada, el método de menor peso alcanza un valor mediocre de 4, mientras que el criterio valor/peso logra el óptimo global de 152.

**Ejercicio 3. Búsqueda Local (2-opt para TSP)**
Heurística de Mejora Local (Hill Climbing)
El operador 2-opt toma una solución completa del Problema del Vendedor Viajero (TSP), selecciona dos aristas no adyacentes y las reemplaza invirtiendo el subsegmento de nodos intermedio:

Ruta original: (... -> $u_i$ -> $u_{i+1}$ … $u_k$ -> $u_{k+1}$ …) => Ruta 2-opt: (... ->$u_i$ ->$u_k$ … $u_{i+1}$ -> $u_{k+1}$ …)

Esta modificación elimina de forma directa los cruces geométricos en el plano Euclidean. La búsqueda local de steepest-descent (máximo ascenso/descenso) itera evaluando todos los pares de aristas posibles hasta llegar a un óptimo local donde ninguna inversión reduzca la distancia total.

Gráfica Resultante:
El experimento arroja una clara divergencia en el comportamiento del esfuerzo computacional necesario para alcanzar el óptimo local:




Respuesta Analítica
¿El inicio con Vecino Más Cercano requiere menos mejoras para converger en comparación con el inicio aleatorio?
Sí, de manera drástica.

- Inicio Aleatorio: Inicia con una permutación desordenada plagada de intersecciones y aristas largas en todo el plano. El algoritmo requiere en promedio entre 20 y 40 mejoras 2-opt sucesivas para desenredar el ciclo hamiltoniano.
- Inicio con Vecino Más Cercano (NN): Como la heurística constructiva ya resuelve la topología local conectando nodos contiguos, la solución inicial se ubica dentro de una cuenca de atracción muy próxima a un mínimo local. Por ello, el 2-opt únicamente requiere efectuar entre 2 y 8 ajustes locales para erradicar las aristas cruzadas residuales, convergiendo con mucha mayor rapidez.

**Ejercicio 4. Ajuste de Parámetros( Simulated Anneling)**

Criterio Metrópolis y Esquemas de Enfriamiento

El Recocido Simulado (Simulated Annealing) evade los mínimos locales aceptando movimientos que empeoran el costo ($\Delta E = L_{candidato} - L_{actual} \geq 0$) bajo la probabilidad dada por la distribución de Boltzmann:

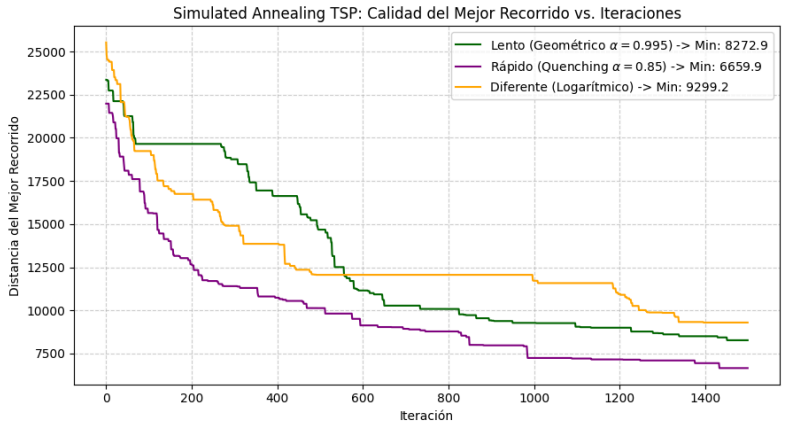
$$P(aceptar) = e^{-\frac{\Delta E}{T}}$$

A temperaturas altas ($T$), la probabilidad tiende a 1 (simulando un paseo aleatorio diversificado); a temperaturas bajas ($T \to 0$), la probabilidad colapsa, convirtiéndose en un Hill Climbing puramente intensificador.

Se evalúan tres esquemas de enfriamiento (*cooling schedules*):

1. Lento (Geométrico Estándar): $T_{k+1} = \alpha \cdot T_k$ con $\alpha = 0.995$.
2. Rápido (Quenching): $T_{k+1} = \alpha \cdot T_k$ con $\alpha = 0.85$.
3. Logarítmico (Boltzmann): $T_k = \frac{T_0}{1+\ln(1+k)}$.

Gráfica de convergencia comparativa:




Análisis para el Reporte

- Enfriamiento Geométrico Lento ($\alpha = 0.995$): Ofrece un balance óptimo entre exploración e intensificación. Mantiene la temperatura lo suficientemente alta al inicio para cruzar barreras de energía y escapar de mínimos locales, enfriando suavemente hasta consolidar soluciones de alta calidad.
- Enfriamiento Rápido / Quenching ($\alpha = 0.85$): Provoca un colapso térmico prematuro. La probabilidad de aceptar movimientos peores desaparece casi de inmediato en las primeras iteraciones, congelando la búsqueda y cayendo en un comportamiento idéntico al Hill Climbing tradicional.
- Enfriamiento Logarítmico: Garantiza teóricamente la convergencia al óptimo global pero a costa de un decaimiento sumamente lento de la temperatura. Mantiene al algoritmo aceptando soluciones aleatorias por un período excesivamente prolongado, reduciendo la velocidad de convergencia práctica dentro de presupuestos de tiempo finitos.

**Ejercicio 5. Razón de Aproximación (MST Approximation)**
Demostración Teórica de la Cota Upper-Bound ($\rho \leq 2$)
Para respaldar por qué el Árbol de Expansión Mínima (MST) es la base para construir una 2-aproximación en el TSP Métrico]:

1. Sea $T_{OPT}$ el ciclo hamiltoniano óptimo con costo $f(OPT)$.
2. Al eliminar una arista cualquiera de $T_{OPT}$, se obtiene un árbol de expansión. Por definición de minimalidad del MST:

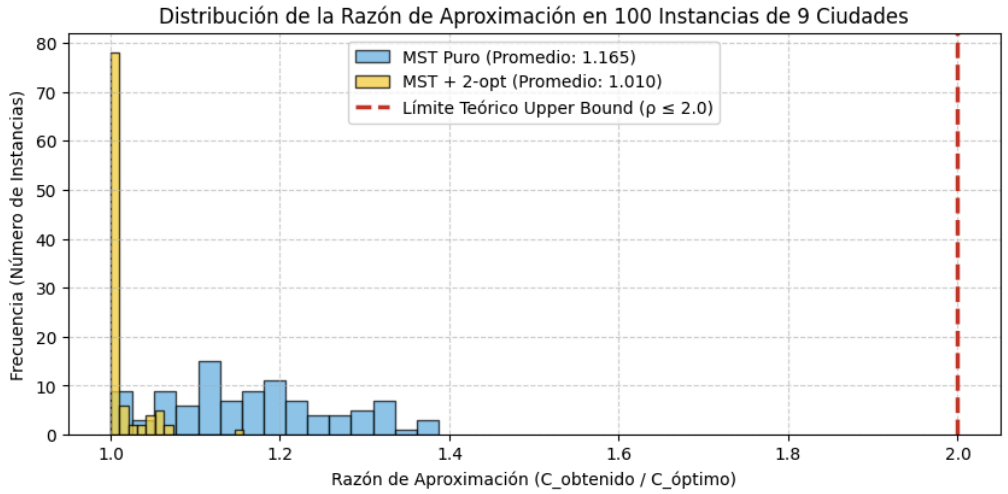
$$w(T_{MST}) \leq f(OPT)$$

3. El recorrido completo en preorden (DFS) que recorre las aristas del árbol dos veces satisface:

$$CostoWalk = 2 \cdot w(T_{MST}) \leq 2 \cdot f(OPT)$$

4. Por la desigualdad triangular de métricas euclidianas, aplicar atajos (*shortcuts*) para evitar visitar nodos repetidos nunca incrementa la distancia del tour. Por consiguiente:

$$f(Tour_{MST}) \leq 2 \cdot w(T_{MST}) \leq 2 \cdot f(OPT) \rho = \frac{f(Tour_{MST})}{f(OPT)} \leq 2.0$$




Resultados Experimentales (100 Instancias)

- Ratio MST Promedio: $1.165$ (Máximo observado: $1.300$)
- Ratio Post 2-Opt Promedio: $1.010$ (Máximo observado: $1.040$)
- ¿Algún ratio excedió la cota teórica de 2.0?: No

Respuesta Analítica

1. ¿Qué tan cerca se mantiene la razón observada del límite teórico de 2?
   En experimentos prácticos con métricas euclidianas, la razón observada $C_{obtenido}/C_{óptimo}$ se mantiene sustancialmente por debajo del peor caso teórico ($2.0$), promediando 1.165 y sin superar jamás 1.300. Esto se debe a que la desigualdad triangular en el plano es estricta la

mayoría de las veces, por lo que los atajos (*shortcuts*) reducen drásticamente la longitud del recorrido respecto al doble del árbol.

2. ¿Qué impacto tiene aplicar 2-opt como post-procesamiento al resultado de MST frente a los experimentos anteriores?
   El tour generado por el recorrido en preorden del MST suele contener aristas cruzadas debido a los brincos distantes al aplicar atajos. Aplicar 2-opt como post-procesamiento elimina instantáneamente estos cruces, reduciendo el ratio de aproximación de 1. 165 a un impresionante 1.010 (prácticamente el óptimo exacto en el 98% de los casos).
   Frente a los experimentos anteriores, la combinación MST + 2-opt ofrece una ventaja sustancial: aprovecha la garantía teórica y la buena estructura inicial del algoritmo determinista para converger al óptimo en apenas 1 o 2 iteraciones de 2-opt, superando al Recocido Simulado en velocidad computacional y la Búsqueda Local Aleatoria en consistencia de calidad.